EXPERIMENT NO. 2

AIM

To create and implement a basic Blockchain using Python by constructing blocks, generating cryptographic hashes, linking blocks using previous hashes, and validating the integrity of the blockchain.

OBJECTIVES

1. To understand the basic structure and working of a Blockchain.
2. To create a Block using Python and store relevant block information.
3. To generate a SHA-256 hash for each block.
4. To link each block with the hash of the previous block.
5. To create a Genesis Block and add subsequent blocks to the blockchain.
6. To implement basic Blockchain validation and detect data tampering.

THEORY

1. Blockchain

A Blockchain is a decentralized system used to record and maintain information in a secure and organized manner. The information is stored in blocks, and each block is connected to the block created before it. Every block contains important details such as data, creation time, the hash of the previous block, and its own hash. This connection makes it difficult to modify information without being detected.

In this experiment, a basic Blockchain is developed using Python to understand how blocks are created, connected, stored, and verified.

2. Structure of a Block

A block in a Blockchain consists of the following components:

- Index: Identifies the position or sequence number of the block.
- Timestamp: Records the date and time when the block was generated.
- Data: Contains the information or transaction details stored in the block.
- Previous Hash: Stores the hash generated by the preceding block.
- Hash: Represents the unique digital fingerprint of the current block.
- Nonce: A number that may be modified during the mining process to satisfy a specific hashing condition.

3. Hashing in Blockchain

Hashing is a process of converting input information into a fixed-size sequence of characters. Blockchain systems commonly use cryptographic hashing algorithms such as SHA-256 to protect the integrity of stored information.

The hash of a block is generated using important block attributes including its index, timestamp, data, previous hash, and nonce.

Block Information → SHA-256 Algorithm → Hash Value

The generated hash provides a unique representation of the block. If even a small portion of the block's information is modified, a completely different hash is produced.

4. Previous Hash and Chain Formation

Blocks in a Blockchain are connected through the previous hash field. Each block stores the hash of the block immediately before it, creating a continuous relationship between blocks.

Block 0 → Block 1 → Block 2 → Block 3 → …

For example, if the data stored in Block 1 is modified, its hash will change. Block 2 will still contain the original hash of Block 1, causing the connection between Block 1 and Block 2 to become invalid.

The linking process can be represented as:

Previous Block Hash + Current Block Information → Current Block Hash

5. Genesis Block

The Genesis Block is the initial block of a Blockchain. Since there is no block before it, its previous hash is assigned a predefined value. A value such as 0, 0000, or a fixed string may be used for this purpose.

The Genesis Block acts as the starting point of the entire Blockchain. Every block created after it is connected directly or indirectly to this first block.

6. Adding a New Block

To add a new block, the Blockchain first identifies the most recently added block. The hash of that block is then assigned to the previous_hash field of the new block. After including the required information, a new hash is generated for the block.

The general process is:

7. Create a new block.
8. Record the block data and current timestamp.
9. Obtain the hash of the latest block.
10. Assign it as the previous hash of the new block.
11. Generate the SHA-256 hash of the new block.
12. Add the newly created block to the Blockchain.

7. Blockchain Validation

Blockchain validation is used to determine whether the blocks have remained unchanged and whether all blocks are properly connected. During validation, the hash stored in each block is compared with a newly generated hash.

A simple validation process checks:

- The calculated hash is equal to the stored hash of the block.
- The previous_hash of a block matches the hash of the block before it.
- The Genesis Block exists as the first block of the Blockchain. ● The sequence of blocks has not been broken or altered.

8. Tampering Detection

Blockchain can identify unauthorized changes through its hashing mechanism. When the contents of an existing block are changed, its original hash is no longer valid. Since the following block contains the old hash as its previous_hash, the connection between the blocks becomes inconsistent.

Therefore, checking the hashes and links between blocks helps detect whether the Blockchain has been tampered with.

ALGORITHM

1. Start the program.
2. Import Python modules required for Blockchain implementation, such as hashlib and time.
3. Define a Block class containing index, timestamp, data, previous hash, hash, and nonce.
4. Define a method to generate the SHA-256 hash of a block.
5. Initialize the Genesis Block using a predefined previous hash value.
6. Create a Blockchain list and insert the Genesis Block.

7. Define a function for generating and inserting new blocks.
8. Obtain the hash of the most recent block and assign it to the previous_hash field.
9. Generate the hash of the newly created block and store it.
10. Add the new block to the Blockchain list.
11. Define a function to verify the integrity and connections of all blocks.
12. Display the stored blocks along with their details.
13. Display whether the Blockchain is valid or has been modified.
14. Stop the program.

FLOW OF BLOCKCHAIN CREATION

Start → Create Genesis Block → Generate Hash → Initialize Blockchain → Create New Block → Assign Previous Hash → Generate Current Hash → Add Block → Verify Blockchain → Display Result → Stop

PROGRAM 1: CREATE A BLOCK AND GENERATE ITS HASH

Write a Python program to define a Block and calculate its SHA-256 hash using the block's index, timestamp, data, previous hash, and nonce.

CODE

```
import hashlib

class Block:

   def __init__(self, index, timestamp, data, previous_hash, nonce=0):
      self.index = index
self.timestamp = timestamp
      self.data = data
      self.previous_hash = previous_hash
      self.nonce = nonce

      self.hash = self.calculate_hash()

   def calculate_hash(self):
      block_string = (
str(self.index) +
self.timestamp +          self.data
+          self.previous_hash +
str(self.nonce)
      )

      return hashlib.sha256(
         block_string.encode("utf-8")
      ).hexdigest()



block = Block(
   1,
   "2026-08-28 14:30:00",
   "Alice pays Bob $100",
   "0"
)

print("Index:", block.index) print("Data:",
block.data)
```

```
print("Previous Hash:", block.previous_hash)
print("Hash:", block.hash)
```

OUTPUT

```
Index: 1
Data: Alice pays Bob $100
Previous Hash: 0
Hash: b8890b32ebf76c9604092cb1cf10083536f68a6a2bb566180bfbe343e757f19c

=== Code Execution Successful ===
```

PROGRAM 2: CREATE A BLOCKCHAIN

Write a Python program to create a simple Blockchain containing a Genesis Block and additional blocks. Display the details of each block, including its index, data, previous hash, and current hash.

CODE

```
import hashlib


class Block:

  def __init__(self, index, timestamp, data, previous_hash, nonce=0):
    self.index = index
self.timestamp = timestamp
    self.data = data
    self.previous_hash = previous_hash
    self.nonce = nonce

    self.hash = self.calculate_hash()

  def calculate_hash(self):
    block_string = (
str(self.index)          +
self.timestamp
        + self.data
        + self.previous_hash
        + str(self.nonce)
    )

    return hashlib.sha256(
block_string.encode("utf-8")
    ).hexdigest()



class Blockchain:

  def __init__(self):
self.chain = [
      Block(
        0,
        "2026-08-28 14:30:00",
        "Genesis Block",
        "0"
      )
    ]

  def add_block(self, data, timestamp):
    previous = self.chain[-1]

    new_block = Block(
len(self.chain),          timestamp,
      data,
      previous.hash
    )

    self.chain.append(new_block)



blockchain = Blockchain()
```

```
blockchain.add_block(
"Alice pays Bob $100",
   "2026-08-28 14:30:01"
)

blockchain.add_block(
"Bob pays Charlie $50",
   "2026-08-28 14:30:02"
)


for block in blockchain.chain:     print("Block",
block.index)     print("Data:", block.data)
print("Previous Hash:", block.previous_hash)
   print("Hash:", block.hash)
print()
```

OUTPUT

```
Block 0
Data: Genesis Block
Previous Hash: 0
Hash: 435b752fe81b1d7a9358f9ce57236298362701bc19f766df649a1752c1f34388

Block 1
Data: Alice pays Bob $100
Previous Hash: 435b752fe81b1d7a9358f9ce57236298362701bc19f766df649a1752c1f34388
Hash: de1d39d25a5cb8ff6619d8da71bfc503307428cef5d7c2d7a4adfdcd8055af6c

Block 2
Data: Bob pays Charlie $50
Previous Hash: de1d39d25a5cb8ff6619d8da71bfc503307428cef5d7c2d7a4adfdcd8055af6c
Hash: d0ec5513a9172f689e20ca4030ae8c9c61e06e5c29a01db9d1fc49a87b9e4264


=== Code Execution Successful ===
```

PROGRAM 3: VALIDATE THE BLOCKCHAIN

Write a Python program to validate the Blockchain by checking whether the hash stored in every block is correct and whether each block correctly points to the previous block.

CODE

```
import hashlib class Block:
    def __init__(self, index, timestamp, data, previous_hash, nonce=0): self.index = index
        self.timestamp = timestamp self.data = data self.previous_hash = previous_hash
        self.nonce = nonce self.hash = self.calculate_hash()


    def calculate_hash(self): block_string = ( str(self.index) + self.timestamp +
            self.data + self.previous_hash + str(self.nonce)
        )
        return  hashlib.sha256(block_string.encode("utf-8")).hexdigest()



class Blockchain:
    def _init_(self): self.chain = [
            Block(0, "2026-08-28 14:30:00", "Genesis Block", "0")
        ]


    def add_block(self, data, timestamp): previous = self.chain[-1]
        self.chain.append(
            Block(len(self.chain), timestamp, data, previous.hash)
        )


    def is_valid(self):
        for i in range(1, len(self.chain)): current = self.chain[i]
            previous = self.chain[i - 1]
if current.hash != current.calculate_hash(): return False
if current.previous_hash != previous.hash: return False
return True



blockchain = Blockchain()
blockchain.add_block("Alice pays Bob $100", "2026-08-28 14:30:01") blockchain.add_block("Bob pays Charlie $50",
"2026-08-28 14:30:02") print("Blockchain valid:", blockchain.is_valid())

# Demonstrate tampering detection blockchain.chain[1].data = "Alice pays Bob
$500" print("After tampering:", blockchain.is_valid())
```

OUTPUT

```
Blockchain valid: True
After tampering: False

=== Code Execution Successful ===
```

OBSERVATION

The Blockchain consists of multiple blocks connected through cryptographic hashes. Each block contains the hash of the previous block, creating a secure chain. Any change in the data of a block changes its hash and can be detected during validation.

RESULT

The basic Blockchain was successfully created using Python. Blocks were constructed with relevant information, SHA-256 hashes were generated, blocks were linked using previous hashes, and the integrity of the Blockchain was checked using validation.

CONCLUSION

The experiment demonstrated the fundamental working of Blockchain technology using Python. A Blockchain is formed by linking blocks through cryptographic hashes. The Genesis Block provides the starting point, while every subsequent block stores the hash of its predecessor. SHA-256 hashing provides data integrity and makes unauthorized modification detectable. The validation process further demonstrates how the consistency of the Blockchain can be verified.